Control Engineering Practice 33 (2014) 84–93

Contents lists available at ScienceDirect

Control Engineering Practice

journal homepage: www.elsevier.com/locate/conengprac

Formal representation of GRAFCET to automatically generate control code

Frank Schumacher, Alexander Fay*

Helmut-Schmidt-University, Institute of Automation Technology, Holstenhofweg 85, 22043 Hamburg, Germany

ARTICLE INFO

Article history:
Received 17 April 2014
Accepted 10 September 2014
Available online 11 October 2014

Keywords:
Modeling language
GRAFCET
Formalization
Control code
Programmable logic controller

ABSTRACT

Within the engineering of control code, model-based approaches become more and more important. Basically GRAFCET serves as a suitable modeling language but lacks an exhaustive formal representation of its structure and dynamic behavior since current approaches only consider subclasses of GRAFCET. Within this article the authors depict a systematic approach for an exhaustive formal model of GRAFCET, specifically by taking *enclosing steps*, *forcing orders* and *time constraints* into account. Based on this formal model of GRAFCET a method together with a tool framework are presented allowing for the automatic generation of control code.

© 2014 Elsevier Ltd. All rights reserved.

1. Introduction

1.1. Engineering of programmable logic controllers today

Regarding the work of a control engineer today, the development of an industrial automation solution is mostly achieved by direct implementation of the control task based on the interpretation of informal text documents (*informal specification*). This effort is assisted by engineering tools, e.g. for the programming of Programmable Logic Controllers (PLC). However, especially in industrial automation, where unique specifications for every machine or plant exist, the requirements of the control software have to be manually transferred into the control program as they are not formally defined in practice due to a lack of time and expertize. This practice (Johnson, 2007) most often leads to a deficient documentation of sequential interdependencies within the control program and additional costs caused by the erroneous interpretation of the textual requirements.

Since nowadays the design of logic controllers can be considered as a software project, modeling languages and appropriate methods from the software engineering domain have been proposed to be adapted to the automation engineering domain, such as the paradigm of model-driven development (MDD) (Thramboulidis & Frey, 2011). To emphasize the value-adding activities, the automatic generation of control code from formal design models plays a key role for supporting the idea of MDD in the automation engineering domain. Though having great opportunities and a wide acceptance among researchers (Witsch & Vogel-Heuser, 2009; Lukman, Godena, Gray, Hericko, & Strmcnik, 2013), most of those methods and tools still lack acceptance in industrial practice. The reason is twofold: on the one hand formal methods proposed in the literature are based on modeling languages which are usually not familiar to the designated users (control practitioners). On the other hand, most model-driven approaches are not feasible in practice since they only allow performing modifications and revisions within the (formal) models. However, practical experience of the PLC engineering shows that the majority of modifications are directly implemented within the PLC code and thus need to be redocumented into the corresponding specification. As a consequence, formal approaches are not well accepted in practice. Furthermore, he controller's formal design and specification is often neglected in industrial practice by reason of time pressure, the necessity of transforming a (graphical) design model into PLC code manually (or vice versa), and the lack of tool support.

1.2. Related work

The authors propose to use GRAFCET according to IEC 60848 (IEC 60848, 2013) as a basic principle. GRAFCET is a graphical modeling language for the design of the controller's dynamic behavior. It is widely used in France, where it has been conceptualized, and is increasingly used in other countries. In 2005, it became an integral part of the professional education of control practitioners on the level of technicians in Germany. Thus, GRAFCET can be an adequate basis for a systematic introduction of formal methods within the automation engineering domain. But even if GRAFCET is used as a modeling language within the design phase of PLC code, it still has to be

* Corresponding author. Fax: +49 406 541 2004.
E-mail addresses: frank.schumacher@hsu-hh.de (F. Schumacher), alexander.fay@hsu-hh.de (A. Fay).

http://dx.doi.org/10.1016/j.conengprac.2014.09.008
0967-0661/© 2014 Elsevier Ltd. All rights reserved.

translated manually into one of the IEC 61131-3 programming languages (IEC 61131-3, 2013) since there is neither a transformation method nor an appropriate tool support based on a vendor independent formal interpretation of the GRAFCET's structure and dynamic behavior.

The missing exhaustive formal definition of the GRAFCET's structure and dynamic behavior is a significant hindrance towards model-driven development within the automation technology domain. Therefore, many researchers have spent efforts on finding the right formalism for the definitions given in the IEC 60848 standard. In one direction of this research, finite state machines serve as a formal model of GRAFCET. These approaches, such as (Roussel & Lesage, 1996), predominantly tend to formal analysis of the control algorithm's specification to guarantee certain properties, like e.g. liveness or safety. Furthermore, they facilitate the synthesis of the control algorithm (Carré-Ménétrier, & Zaytoon, 2002) and the automatic generation of black box test sequences (Provost, Roussel, & Faure, 2010). In summary, the formal finite state machine model aims at representing the dynamic behavior of GRAFCET and prescinds from its original structure. Following a different direction of research, GRAFCET can be interpreted in terms of a control interpreted Petri net (CIPN) (David & Alla, 2010) and thus the mathematical formalism of Petri nets can be applied to GRAFCET to describe its structure and its dynamic behavior. Accordingly, it is reasonable to apply the formalism of CIPN for an exhaustive description of GRAFCET.

The lack of a suitable exhaustive formal representation as well as the lack of a method for an automatic generation of PLC code is illustrated by an overview of related work in Table 1.

In summary, current approaches only consider subclasses of GRAFCET within their formal model. In most cases, these subclasses only contain the very basic elements of a non-hierarchical and non-time dependent GRAFCET graph (*Grafcet*), which is named the *Basic-Grafcet* (David & Alla, 2010). There is no formal model which covers all elements defined in IEC 60848. Additionally, an appropriate method and tool support for automatically generating PLC code compliant with IEC 61131-3 definitions has not yet been established.

Therefore, the authors intend to define an exhaustive formal model of GRAFCET, especially taking *enclosing steps*, *forcing orders* and *time constraints* into account, and to develop a method which allows for the automatic generation of PLC code.

1.3. Outline of the article

Chapter 2 reflects the formal model of a *Basic-Grafcet*. Subsequently, an approach to extend the *Basic-Grafcet's* formalism by means of hierarchical structuring, especially *enclosure* and *forcing orders*, and *time dependencies* is presented in Chapter 3. This results in a formal model which covers all GRAFCET elements according to IEC 60848 definitions and serves as a starting point for the automatic generation of PLC code. Chapter 4 is devoted to the developed method for a computer-based transformation of *Grafcets* into PLC code and depicts a software tool framework which assists the control practitioner in using GRAFCET for the specification of logic controllers and offers an automatic generation of PLC code. The approach is illustrated by means of an industrial size application example in Chapter 5. Chapter 6 reviews the main results presented within the contribution and concludes the paper by giving a short discussion of further research needs.

2. The formal model of a Basic-Grafcet

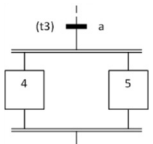
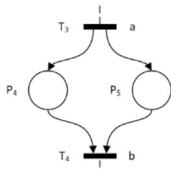
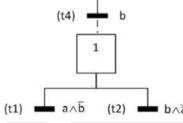
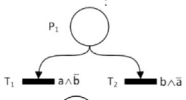
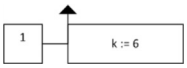
2.1. Structure of a Basic-Grafcet

Originally examined in (David, 1995), *Basic-Grafcets* can be seen as a specific kind of Petri net, a Control Interpreted Petri net (CIPN). Since

Table 1
Overview of related work.

Publication	Alvarez et al. Alvarez, Burgos, Sarachaga, Estévez, and Marcos (2012)	Arnold et al. Arnold and Henriques (2005)	Carré-Ménétrier et al. Carré-Ménétrier and Zaytoon (2002)	Charbonnier et al. Charbonnier, Alla, and David (1995)	Chériaux et al. Chériaux, Picci, Provost, and Faure (2010)	Frensel et al. Frensel and Bruijn (1998)	Gaertner et al. Gaertner and Thirion (1999)	L'Her et al. L'Her, Le Parc, and Marcé (1995)	Le Parc et al. Le Parc, L'Her, Scharbarg, and Marcé (1999)	Mallet et al. Mallet, Gaffe, and Boeri (2000)	Marcé et al. Marcé, L'Her, and Le Parc (1996)	Martinez et al. Martínez and Martínez (2005)	Philippot et al. Philippot and Tajer (2010)	Provost et al. Provost et al., (2010)	Roussel et al. Roussel and Lesage (1996)
GRAFCET elements considered within the formal model															
Forcing orders															
Enclosing steps															
Macro-steps					X									X	
Time constraints							X		X		X				
Stored actions	X	X		X	X				X					X	X
Continuous actions	X	X	X	X	X	X	X		X	X		X	X	X	X
Basic-Grafcet	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
Purpose															
PLC code generation	X														
Code generation		X					X			X		X			
Control synthesis			X	X									X		
Validation/test					X									X	X
Verification						X		X	X		X				X

Table 2
Graphical elements of a *Basic-Grafcet* and their equivalent CIPN elements.

	GRAFCET	CIPN
Step, transition, junction		
Initial step		determined through initial marking
Simultaneous sequences divergence/ convergence		
Selection divergence		
Continuous action		
Stored action		

both modeling languages can be traced back to Place/Transition nets, their formal definitions are the same. This offers the opportunity to apply well known formal methods from the Petri net domain to GRAFCET. Concerning this matter, Table 2 shows an overview of the *Basic-Grafcet's* elements and their equivalent CIPN elements. As proposed in David and Alla, (2010) a sound and correct *Basic-Grafcet* behaves exactly like a safe CIPN and can be considered as a starting point for an exhaustive formal model of GRAFCET according to IEC 60848 definitions. This is the precondition for the authors' contribution on the extended formal model of GRAFCET, presented in chapter 3. Furthermore, the formalization is the precondition for the application of consistent formal methods to GRAFCET, such as unambiguous code generation.

Accordingly, a *Basic-Grafcet* is defined as "the normalized representation of the main elements" (David & Alla, 2010). This includes steps, transitions, junctions, Boolean transition conditions under consideration of (Lesage, Roussel, & Thierry, 1996), unconditional continuous actions and stored actions. Thus a *Basic-Grafcet* $\boldsymbol{G}_{\text{basic}}$ can be described as a 9-tupel $\boldsymbol{G}_{\text{basic}}=(\boldsymbol{S}, \boldsymbol{T}, \boldsymbol{W}, \boldsymbol{sit}_0, \boldsymbol{I}, \boldsymbol{O}, \boldsymbol{r}, \boldsymbol{a}, \boldsymbol{\Omega})$ consisting of two different sets of nodes, the set of steps $\boldsymbol{S}$

$$\boldsymbol{S}=\{s_1,\ldots,s_i\},\ i=1../S/ : \text{finite number of steps, including macro}-\text{steps} \tag{1}$$

And the set of transitions $\boldsymbol{T}$

$$\boldsymbol{T}=\{t_1,\ldots,t_j\},\ j=1.../\boldsymbol{T}/ : \text{finite number of transitions.} \tag{2}$$

Each step $s_i \in \boldsymbol{S}$ possesses a logical state (active or inactive) which is described by a Boolean variable, the step variable X_i. X_i is allocated to every step, respectively, whereas *true* ($X_i=1$) represents the active step state and *false* ($X_i=0$) the inactive state of the step. The current situation $\boldsymbol{sit}(m)$ at time m within the evolution of a GRAFCET arises from the set of active steps $\boldsymbol{sit}(m)=\{X_1(m),\ldots, X_{/\boldsymbol{S}/}(m)\}$. $\boldsymbol{sit}(m)$ can be compared to the marking of a Petri net at time m. The initial situation $\boldsymbol{sit}_0$ is determined as $\boldsymbol{sit}(m=0)$ and graphically emphasized through a double framing of related steps.

Steps and transitions are linked by junctions. A junction always connects two elements of different sets of nodes in such a way that it only links steps with transitions and transitions with steps. The set of steps directly preceding a transition is defined as $\bullet t_j=\{s_i \in \boldsymbol{S} \mid (s_i,\ t_j) \in \boldsymbol{W}\}$. The set of steps directly succeeding a transition is defined as $t_j\bullet=\{s_i \in \boldsymbol{S} \mid (t_j,\ s_i) \in \boldsymbol{W}\}$. The set of transitions directly preceding or succeeding a step can be defined $\bullet s_i$ and $s_i\bullet$ in an analogous way. All junctions are grouped within the flow relation $\boldsymbol{W}$ of the *Basic-Grafcet*, such that

$$\boldsymbol{W}=\boldsymbol{S}\times\boldsymbol{T}\cup\boldsymbol{T}\times\boldsymbol{S} \tag{3}$$

Macro-steps are only considered implicitly within the formal model. Macro-steps allow the composition and decomposition of a *Grafcet*. On the one hand, if a macro-step expansion is not specified yet, a macro-step is a placeholder and will be considered a "normal" step. On the other hand, if a macro-step expansion exists, the macro-step will be replaced by its macro-step expansion, such as macro-Step M1 in Fig. 1.

The interpretation-part of a *Basic-Grafcet* consists of a finite set of inputs $\boldsymbol{I}$ and a finite set of outputs $\boldsymbol{O}$, two mapping functions $\boldsymbol{r}$ and $\boldsymbol{a}$, and an output function $\boldsymbol{\Omega}$. A receptivity $r(t_j)$, composed of an event $\boldsymbol{E}$ and a Boolean condition $\boldsymbol{B}_{\text{bool}}$, is associated to each transition $t_j \in \boldsymbol{T}$ and summarized in the finite set of receptivities $\boldsymbol{r}$ such that

$$r(t_j)=\boldsymbol{E}_j\wedge\boldsymbol{B}_{\text{bool},j} \tag{4}$$

As the IEC 60848 standard defines, each step can be associated with a finite set of actions, each of them manipulating one output of $\boldsymbol{O}$. The actions of a *Basic-Grafcet*, which can be either *stored actions* $\boldsymbol{a}_{\text{store}}$ or *continuous actions* $\boldsymbol{a}_{\text{cont}}$, are summarized in the set of actions $\boldsymbol{a}$

$$\boldsymbol{a}=\boldsymbol{a}_{\text{store}}\cup\boldsymbol{a}_{\text{cont}},\ \boldsymbol{a}_{\text{store}}\cap\boldsymbol{a}_{\text{cont}}=\varnothing \tag{5}$$

A *stored action* $a_{\text{store}}(s_i)$ is defined as an operation on a numerical or Boolean variable that is carried out once at occurrence of a defined event. *Continuous actions* $a_{\text{cont}}(s_i)$ manipulate Boolean variables if the associated step is in active state and the optional condition $Cond_{\text{bool}}(a_{\text{cont}})$ is fulfilled. $Cond_{\text{bool}}(a_{\text{cont}})$ is a Boolean term which may be associated to the continuous action as an additional execution condition. Dependent on $\boldsymbol{sit}(m)$ only a subset of actions in $\boldsymbol{a}$ is executed. Therefore the output function $\boldsymbol{\Omega}$ contains all executable actions with regard to $\boldsymbol{sit}(m)$.

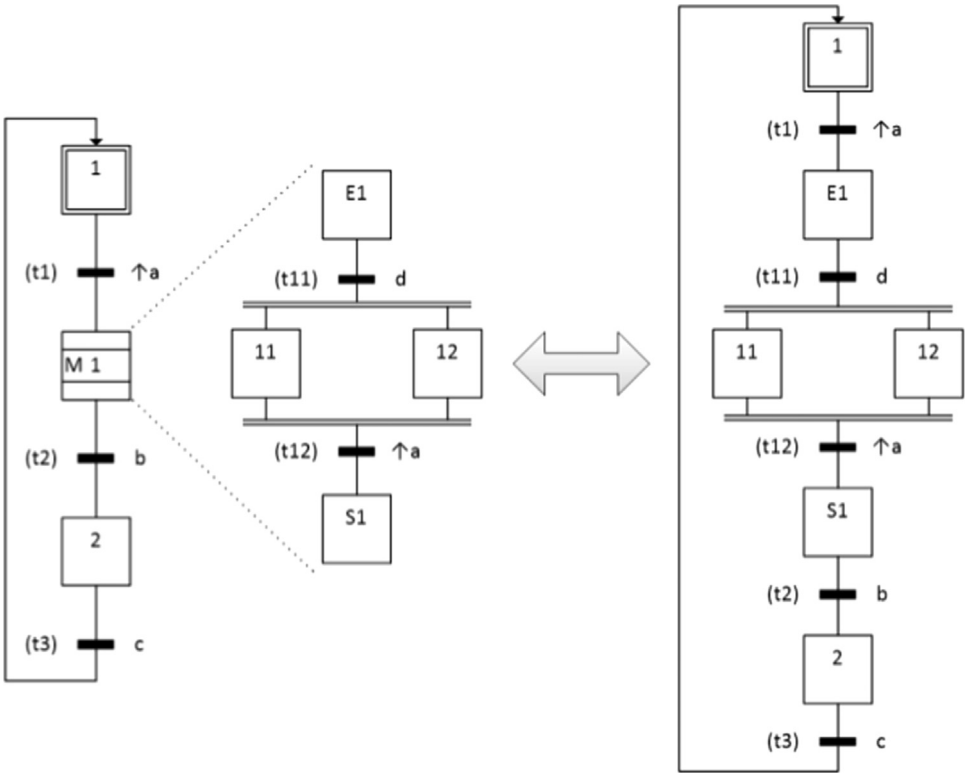


Fig. 1. Principle of decomposition by use of a GRAFCET macro-step.

2.2. Dynamic behavior of a Basic-Grafcet

For the calculus of the *Basic-Grafcet's* dynamic behavior the IEC 60848 standard provides five evolution rules which ensure a deterministic behavior, as follows:

Rule 1: The initial situation, chosen by the designer, is the situation at the initial time.
Rule 2: A transition is *enabled* when all immediately preceding steps are active. A transition is *cleared* when the transition is enabled, and when its associated transition-condition is true.
Rule 3: When a transition is cleared all preceding steps get deactivated and all succeeding steps get activated.
Rule 4: Several transitions, which can be cleared simultaneously, are simultaneously cleared.
Rule 5: An active step which is simultaneously activated and deactivated remains active.

From a formal point of view, the dynamic behavior is described by an instruction for the calculation of the succeeding situation ***sit*** $(m+1)$ based on ***sit***(m). In case of GRAFCET, the definition is that ***sit***$(m+1)$ is calculated in terms of the vector $\mathbf{sit}(m+1)=[X_1(m),\ldots, X_i(m)]^T$ by application of a /***S***/ *x* /***T***/ matrix $\mathbf{C}_{basic}$, which is called the incidence matrix of $\boldsymbol{G}_{basic}$:

$$\mathbf{sit}(m+1)=\mathbf{sit}(m)+\mathbf{C}_{basic}\cdots\boldsymbol{\delta}_{basic}(m). \tag{6}$$

The elements c_{ij} of $\mathbf{C}_{basic}$ are determined as $c_{ij}=-1$, if $(s_i,t_j)\in \boldsymbol{S}\ x\ \boldsymbol{T}$, $c_{ij}=1$, if $(s_i,t_j)\in \boldsymbol{T}\ x\ \boldsymbol{S}$ and $c_{ij}=0$, otherwise. The elements δ_j of $\boldsymbol{\delta}_{basic}(m)=[\delta_1(m),\ldots, \delta_j(m)]^T$ are determined as $\delta_j(m)=1$, if transition t_j is cleared and $\delta_j(m)=0$, otherwise. t_j is *cleared* (fires), if it is enabled and the corresponding receptivity $r(t_j)$ is fulfilled. t_j is called *enabled*, if all preceding steps are in active state. Hence the constraint for clearing of a transition t_j at time m is defined

$$\begin{aligned} &clearing_m(t_j)=\text{true}, \quad \text{if}\ \forall s_i\in {}^\bullet t_j | X_i=1 \wedge [r(t_j)]=1,\\ &clearing_m(t_j)=\text{false}, \ \text{otherwise} \end{aligned} \tag{7}$$

When there is no event associated to transition t_j, then t_j is receptive to the always occurring event ε (David, 1995). When there is no condition associated to transition t_j, then t_j is receptive to the always effective Boolean condition *true*.

If there is an element of **sit**$(m+1)$ with the value $X_i<0$ (e.g. $X_i=-1$) it is set to 0, since each step variable X_i is of Boolean value and cannot contain any further information regarding the chronological sequence of the status of the step. If there is an element of **sit**$(m+1)$ with the value $X_i>1$ (e.g. $X_i=2$) it is set to 1 because the evolution rule five of the IEC 60848 standard defines an active step which is part of the current situation m and next situation $m+1$ to remain active.

3. Formalization of additional GRAFCET elements

According to IEC 60848 definitions, the *Basic-Grafcet's* dynamic behavior is equivalent to a safe CIPN. Additional elements for hierarchical structuring, such as *enclosure* and *forcing*, as well as *time constraints* are not considered within this formal model. All these additional GRAFCET elements are compacted expressions (David & Alla, 2010). This means, that the dynamic behavior specified by those elements can be traced back to a *Basic-Grafcet* with equivalent behavior, which is called the *normalized Grafcet*. On this basis, a *normalization*, i.e. an integration of the additional GRAFCET elements into the *Basic-Grafcet's* formal model, is possible. In the following, we apply the principle of *normalization* for the formalization of *enclosure*, *forcing* and *time constraints*. The formalization is the precondition for an unambiguous code generation.

3.1. Formalization of hierarchical elements

3.1.1. Enclosure

An *enclosure* (s_E) (Guéguen & Bouteille, 2001) consists of one enclosing step $s_E \in \boldsymbol{S}$ which affects the status of each enclosed step $s^E \in \boldsymbol{S}$ in a way that activation of s_E results in simultaneous activation of all corresponding s^E. When s_E is deactivated, all steps of the enclosed *Partial-Grafcets* are simultaneously deactivated, too. Hence, from a structural point of view, the *Basic-Grafcet's* 9-tupel is extended by the finite set of enclosing steps $\boldsymbol{S}_E \subseteq \boldsymbol{S}$, and $\boldsymbol{\varphi} = \{\varphi(s_{E1}), \ldots, \varphi(s_{/SE/})\}$ being the finite set of enclosures associated to the enclosing steps.

Regarding the interpretation part, the use of enclosures in GRAFCET influences the flow relation $\boldsymbol{W}$ of the *Grafcet's* formal representation due to implicit junctions. These implicit junctions can be captured in the incidence matrix of enclosures $\mathbf{C}_\varphi$, as stated subsequently:

$$\mathbf{C}\varphi = \begin{bmatrix} c1,1 & c1,2 & c1,10 & c1,11 & c1,12 & c1,10' & c1,11' & c1,12' \\ c2,1 & c2,2 & c2,10 & c2,11 & c2,12 & c2,10' & c2,11' & c2,12' \\ c10,1 & c10,2 & c10,10 & c10,11 & c10,12 & c10,10' & c10,11' & c10,12' \\ c11,1 & c11,2 & c11,10 & c11,11 & c11,12 & c11,10' & c11,11' & c11,12' \\ c12,1 & c12,2 & c12,10 & c12,11 & c12,12 & c12,10' & c12,11' & c12,12' \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \end{bmatrix}$$

by normalization, the abbreviatory nature of an enclosure is revealed by making the implicit junctions explicit. This means that additional junctions and transitions occur due to normalization, as illustrated with an example in Fig. 2. As a result, the incidence matrix of a hierarchically structured *Grafcet* containing *enclosing steps* results from aggregation of $\mathbf{C}_{basic}$ and $\mathbf{C}_\varphi$, according to the definitions in chapter 2.2. Further details about the formalization of enclosures, as well as forcing orders, are given in (Schumacher, Schröck, & Fay, 2013).

Since an *enclosing step* possesses all characteristics of a step (see definitions in chapter 2.1), the hierarchical construct of *enclosure* does not have further effects on the dynamic behavior of a *Grafcet*.

3.1.2. Forcing

Each forcing order $\tau(s_i) \in \boldsymbol{\tau}$ is symbolized through a double framing of an action block that is associated to a step $s_i \in \boldsymbol{S}$. It forces a subordinate *Partial-Grafcet* into a specific situation, which can be

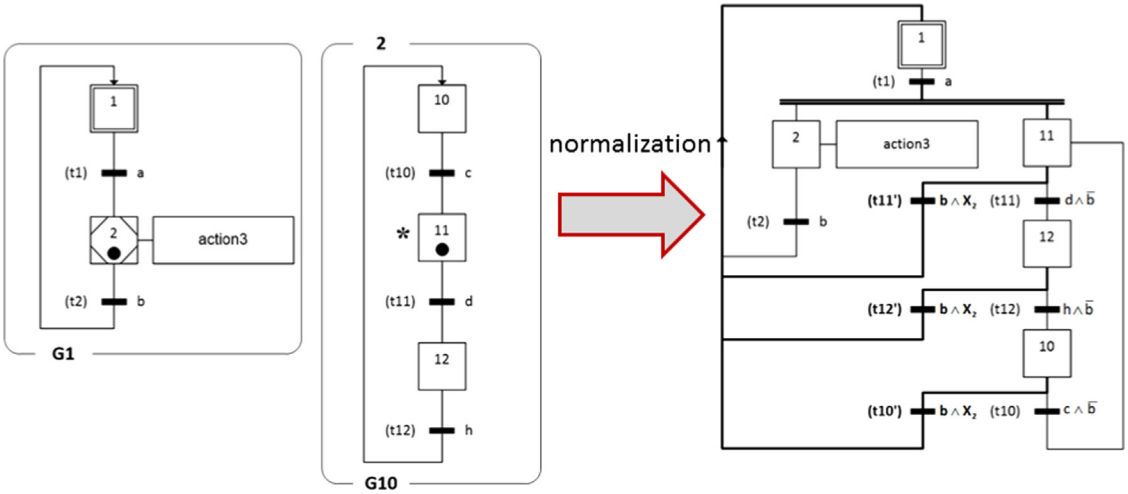


Fig. 2. Principle of normalization using the example of enclosure.

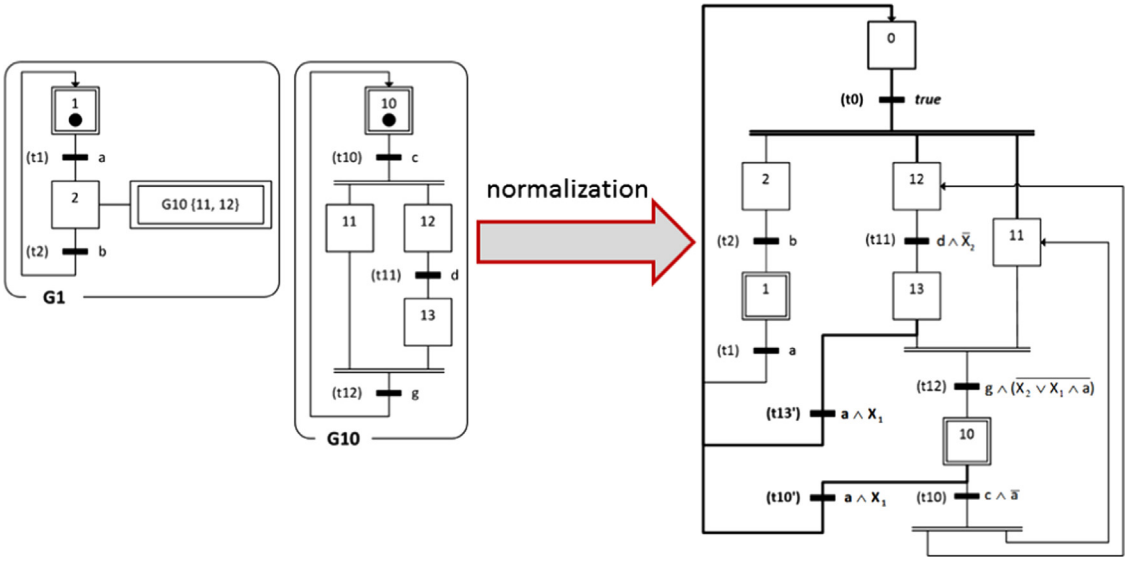


Fig. 3. Principle of normalization using the example of forcing.

- the initial situation of the *Partial-Grafcet*,
- the empty situation (all steps of the *Partial-Grafcet* get inactive of necessity),
- the actual situation,
- a user-defined situation of the *Partial-Grafcet*.

In case of a forcing order, the forced *Partial-Grafcet* cannot evolve from its situation. Hence the forced *Partial-Grafcet* is considered frozen. Altogether, a forcing order can be considered as an internal continuous action extending Eq. (5) by a finite set of forcing orders $\boldsymbol{\tau} \in \boldsymbol{a}$ such that

$$\boldsymbol{a} = \boldsymbol{a}_{\text{store}} \cup \boldsymbol{a}_{\text{cont}} \cup \tau, \boldsymbol{a}_{\text{store}} \cap \boldsymbol{a}_{\text{cont}} \cap \tau = \varnothing \tag{8}$$

Concerning the interpretation part, forcing orders have a great influence on the evolution rules of a *Basic-Grafcet* since a transition t_j cannot be cleared, if a step $s_i \in \bullet t_j$ is part of an active enclosure. This means that forcing orders take priority over the evolution rules of GRAFCET. Hence the constraint for clearing of a transition t_j at time m in (7) has to be extended as follows:

$$\begin{aligned} &clearing_m(t_j)' = \text{true}, \quad \text{if } \forall s_i \in \bullet t_j | (X_i = 1 \wedge \tau(s_i) = 0) \wedge [r(t_j)] = 1 \\ &clearing_m(t_j)' = \text{false}, \text{ otherwise} \end{aligned} \tag{9}$$

Applying the principle of *forcing* (Deneux, 1983) to a hierarchical GRAFCET specification requires an additional semantic interpretation of the forcing order's effects on the dynamic behavior of the *Grafcet*. As described in the case of enclosures, the use of forcing orders in GRAFCET influences the flow relation $\boldsymbol{W}$ of the *Grafcet's* formal representation due to implicit junctions. Assuming that the succeeding situation of a *Partial-Grafcet* after cancellation of a forcing order is calculated from its forced situation, the effects of forcing can be captured in the incidence matrix of forcing orders $\mathbf{C}_F$, as discussed in detail in (Schumacher & Fay, 2013). These additional junctions and transitions occur due to normalization, as presented by an example in Fig. 3.

For all transitions t directly preceding a step s_F, which is associated to a forcing order in terms of $t_F \in \bullet s_F$, there is an implicit junction from t_F to each forced step s^F of the forced *Partial-Grafcet*. If the forcing order associated to s_F advises a step s^F of the subordinate *Partial-Grafcet* to be deactivated, then there is an implicit junction from t_F to s^F of value -1. If the forcing order of s_F advises a step s^F of the subordinate *Partial-Grafcet* to be activated, then there is an implicit junction from t_F to s^F of value 1. All other elements of $\mathbf{C}_F$ are determined 0. According to the syntactical specification given in IEC 60848 and by reason of generality, the *normalized Grafcet* in Fig. 3 has to be expanded by step s_0 and transition t_0. Nevertheless, s_0 and t_0 do not have an effect on the dynamic behavior of the normalized *Grafcet*.

As a result, the incidence matrix of a hierarchically structured *Grafcet* containing *enclosing steps* and *forcing orders* results from aggregation of $\mathbf{C}_{\text{basic}}$, $\mathbf{C}_\varphi$ and $\mathbf{C}_F$.

3.2. Formalization of time dependencies

As the definitions of the IEC 60848 standard show, time constraints are characterized as Boolean conditions which underlie a specific syntax and therefore must be interpreted in a specific way. Time constraints can be applied to receptivities or continuous actions in exactly the same way. The notation of a time constraint in GRAFCET is generally defined as $t_1/\alpha/t_2$, while α is named the *time dependent variable*. Besides the time dependent variable, the delay times t_1 and t_2 ($t_1 < t_2$; $t_1, t_2 \geq 0$) have to be defined to obtain a correct time constraint in GRAFCET. The IEC 60848 standard also allows two abbreviated notations which can be seen as particular cases of $t_1 / \alpha / t_2$. The first abbreviation refers to the case of $t_2 = 0$. Instead of writing $t_1/\alpha/0$ s, the notation is t_1/α. This kind of time constraint specifies a simple time delay. In contrast, a simple time limit is specified by the expression $\neg\ (t_1/\alpha)$, with $\neg$ being the symbol used for the negation of a Boolean expression. The value of the time limit's Boolean variable $[\neg\ (t_1/\alpha)]$ changes from *false* to *true*, when $\uparrow\alpha$ occurs and remains *true* for the time period t_1. At occurrence of event $\uparrow[t_1/\alpha]$, the logical variable $[\neg\ (t_1/\alpha)]$ changes its value from *true* to *false*.

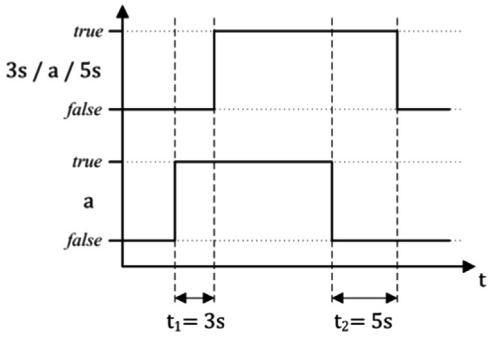


Fig. 4. Notation and interpretation of a time constraint in GRAFCET.

As shown in Fig. 4, a time constraint could e.g. be specified as 3 s/*a*/5 s. This means, that the time constraint's corresponding Boolean variable [3 s/*a* /5 s] changes its value from *false* to *true* three seconds after $\uparrow a$ has occurred. [3 s/ *a*/5 s] changes its value from *true* to *false* not before five seconds after the occurrence of $\downarrow$a. This example illustrates the relationship between the specific time delays (t_1 and t_2) and the two input events ($\uparrow$a and $\downarrow$a) and thus defines a discrete time interval $[t_1..t_2]$, while the logical variable of the time constraint [3 s/ a/5 s] is of value *true*.

From a formal point of view, the time constraint can be modeled as a stop watch θ observing the time dependent variable's change in state. Each time an event occurs, such that $\uparrow\alpha$ or $\downarrow\alpha$, θ is reinitialized from zero and therefore provides information of the elapsed time duration since the last occurrence of $\uparrow\alpha$ or $\downarrow\alpha$. When the elapsed time duration is greater than or equal to t_1 (in case of $\uparrow\alpha$) or t_2 (in case of $\downarrow\alpha$) the corresponding time constraint is met and $[t_1/\alpha/t_2] = 1$. Hence, a *Grafcet* containing time constraints, named timed *Grafcet* $\boldsymbol{G}_{\text{timed}}$, can be defined as

$$G_{\text{timed}} = (\boldsymbol{S}, \boldsymbol{T}, \boldsymbol{W}, \boldsymbol{sit}_0, \boldsymbol{I}, \boldsymbol{O}, \boldsymbol{r}, \boldsymbol{a}, \boldsymbol{\Omega}, \boldsymbol{\Theta}) = (\boldsymbol{G}_{\text{basic}}, \boldsymbol{\Theta}) \tag{10}$$

$\boldsymbol{\Theta}$ is the finite set of stopwatches $\boldsymbol{\Theta} = \{\theta_1, .., \theta_{/\Theta/}\}$ associated to each time dependent variable α. As a consequence the overall state of a *Grafcet* at time m is dependent on $\boldsymbol{sit}(m)$ and the states of the stopwatches.

With regard to the interpretation part of GRAFCET Eq. (4) has to be extended in case of *timed receptivities*, such that

$$r(t_j) = \boldsymbol{E}_j \wedge \boldsymbol{B}_{\text{bool}} \wedge B_{\text{time,j}} \tag{11}$$

The new term $\boldsymbol{B}_{\text{time,j}}$ is a Boolean function which represents the time constraint and can be defined as $\boldsymbol{B}_{\text{time}} = t_1/\alpha$, $\boldsymbol{B}_{\text{time}} = \neg\ (t_1/\alpha)$ or $\boldsymbol{B}_{\text{time}} = t_1/\alpha/t_2$. When there is no time constraint associated to $r(t_j)$, it is considered the always effective time constraint *true*. In case of *timed actions*, continuous actions $a_{\text{cont}}(s_i)$ are in this case executed if the associated step is in active state ($X_i = 1$) and the assignation condition is fulfilled ($[Cond_{\text{bool}}(a^n_{\text{cont}})] = 1$) and the time constraint $\boldsymbol{B}_{\text{time,i}}$ is met ($[\boldsymbol{B}_{\text{time,j}}] = 1$). When there is no time constraint associated to the continuous action of $a_{\text{cont}}(s_i)$, it is considered the always effective time constraint *true*.

Similar to *enclosing steps* and *forcing orders* normalization is also possible in question of *timed receptivities* and *timed actions*. As presented in (Schumacher & Fay, 2013) T-timed Petri nets according to (Ramchandani, 1974) provide a formal basis for time constraints in GRAFCET, since the delay time d_j which is associated to a transition t_j in T-timed Petri nets corresponds to the following

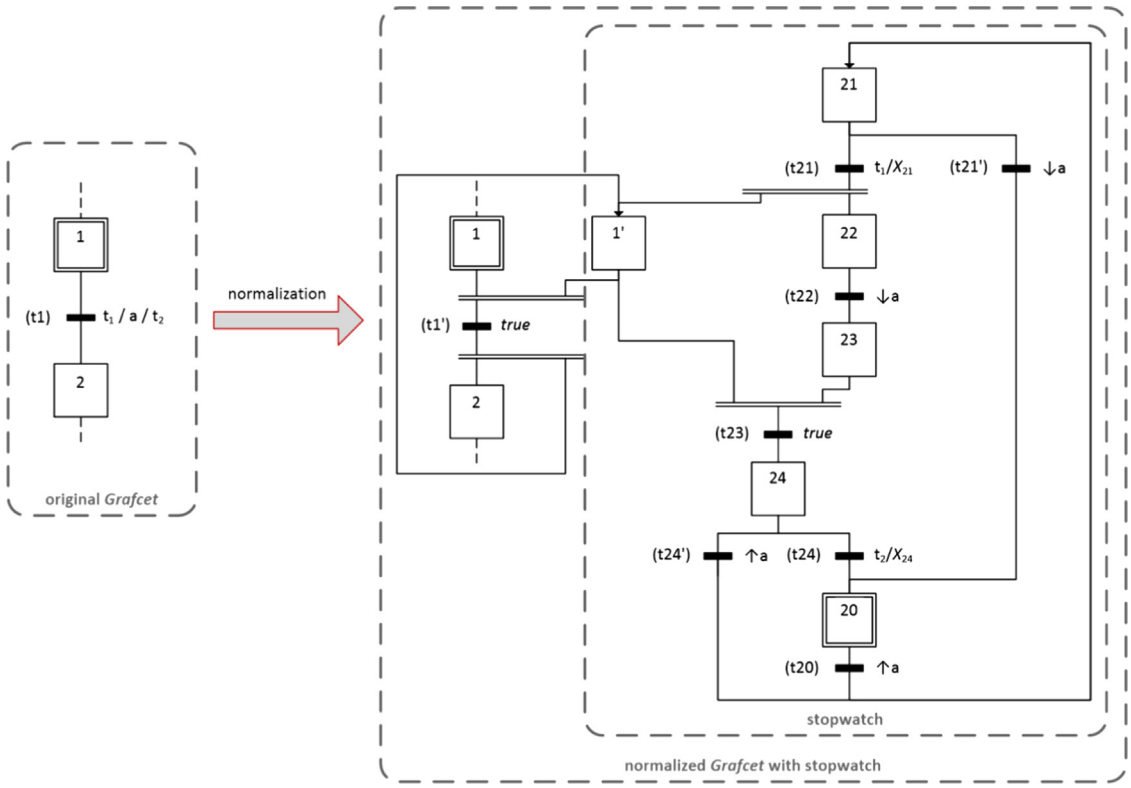


Fig. 5. Timed receptivity and normalized representation.

timed receptivity in GRAFCET:

$$r(t_j) = d_j/X_j/0 = d_j/X_j \tag{12}$$

Here X_j is the step variable of a step $s_i \in {}^\bullet t_j$. Hence, applying the principle of normalization to time constraints results in an additional part of a *Grafcet* which copies the stopwatch behavior by only using elements of the *Basic-Grafcet* and time constraints according to Eq. (12).

Fig. 5 shows an example of a *Grafcet* with one *timed receptivity* and its normalized representation. Linking the stopwatch part to the original Grafcet is achieved through an additional step ($s_{1'}$) while the original timed receptivity of t_1 is replaced by the always effective Boolean condition *true*. To be generic, Fig. 5 includes the transitions $t_{21'}$ and $t_{23'}$. However there are *Grafcets* possible where $t_{21'}$ and $t_{23'}$ can be neglected. In case of *timed actions* the principle shown in Fig. 5 can be applied accordingly. An example of a *timed action* and the normalized form is illustrated in Annex B. Thus, time constraints in GRAFCET can be formalized through adaption of the well-proven formalism of T-timed Petri nets.

4. Towards an automatic generation of PLC code

4.1. Concept for an automatic generation of PLC code

The proposed formal CIPN model allows for a comprehensive and unique interpretation of the GRAFCET's structure and dynamic behavior in an implementation-independent way. In particular, this formalization of GRAFCET is achieved by the principle of normalization, as presented in Chapter 3. This allows the reduction of hierarchical constructs to a *Basic-Grafcet* of equivalent behavior. Considering time dependencies, normalization results in the reduction of timed actions and timed transitions to stopwatches. Based on this, a concept for an automatic generation of PLC code can be established in three steps:

(1) Making the *Grafcet* available in a suitable, standardized data format
(2) Normalization of the *Grafcet's* structure, i.e. the transformation of explicit hierarchical structures into a *Basic-Grafcet*
(3) Transformation of the *normalized Grafcet* into PLC code by application of unambiguous transformation rules

Since the *eXtensible Markup Language* (XML) (World Wide Web Consortium (W3C)) is widely used for data exchange, also in the automation engineering domain, it is reasonable to apply it to the transformation concept in case of GRAFCET. Transformation step (1) offers the opportunity to provide the GRAFCET data in a comprehensible and standardized way, also for other engineering tools. However, the IEC 60848 standard does not offer a suitable XML representation for GRAFCET. As the proposed formal model of GRAFCET is based on CIPN, it is advisable to adapt the *Petri net Markup Language* (PNML), as defined in ISO/IEC 15909-2 (ISO/IEC 15909-2, 2011), for this purpose. The PNML standard is intended to be suitable for any kind of Petri net (Hillah, Kindler, Kordon, Petrucci, & Trèves, 2009), also hierarchically structured Petri nets. But PNML does not support all elements of GRAFCET, such as rising and falling edges of Boolean variables or *enclosing steps* and *forcing orders*. Therefore, as an interim solution, only the *Grafcet's* structure is mapped to PNML directly. All the other GRAFCET elements are mapped to PNML by declaring them to be “toolspecific”. Thus, this approach is in complete consistency with the ISO 15909-2 definitions.

Since the IEC 61131-3 does not offer comparable possibilities to consider hierarchy in the way IEC 60848 does, transformation step (2) is a necessary step. By application of normalization, according to the formal definitions given in the previous chapter, the hierarchically

structured *Grafcet*, possibly consisting of different *partial Grafcets*, is reduced to only one *normalized Grafcet*.

In transformation step (3), transformation rules for the mapping of IEC 60848 constructs into IEC 61131-3 constructs are defined which assure an equivalent behavior of specification (GRAFCET) and implementation (PLC code). Within the presented approach, the focus is on PLC code written in SFC since GRAFCET and SFC share diverse similarities (Schumacher & Fay, 2011). Their common ground is used to a maximum degree for the definition of the transformation rules. But the different possible semantic descriptions of SFCs (Hellgren, Fabian, & Lennartson, 2005) have to be considered as well to guarantee a high degree of interoperability (Fay, Biffl, Winkler, Drath, & Barth, 2013) regarding the engineering tools. Therefore, only such elements of SFC have been taken into account within the transformation rules, which offer an unambiguous interpretation (Bauer, Huuck, Lukoschus, & Engell, 2004). Particularly with regard to the action qualifiers in SFC, only N, S, R, D and L are used in the transformation rules because the dynamic behavior implied in these is vendor-independent. P1 and P0 are not used, since the dynamic behavior implied depends on the final scan logic (IEC 61131-3, 2013; Hellgren et al., 2005) implemented and is therefore a vendor-dependent issue.

Thus, a direct transformation is possible in some cases, as described in Table 3. Nevertheless, in the majority of cases such a simple transformation is not possible and additional declarations have to be made, as shown in Annex A. In cases of non-trivial transformation rules, additional declarations are necessary to assure equivalent dynamic behavior of GRAFCET and SFC according to the definitions given in the standards IEC 60848 and IEC 61131-3. Altogether 28 transformation rules have been defined to assure a correct transformation of all normalized GRAFCET elements to SFC, see Annex A.

Note that since a *Grafcet* may consist of more than one initial step and for a SFC program only one initial step is allowed, the number of the *Grafcet's* initial steps has to be reduced at the end of transformation step (3). Concerning this matter a pragmatic solution is that one initial step is added to the *normalized Grafcet* and is linked to the original initial steps by one transition, whose receptivity is constantly *true*.

Table 3
Example of a direct transformation.

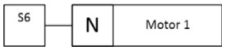
GRAFCET	SFC
	

Table 4
Transformation results in PNML and PLCopenXML.

PNML (GRAFCET)

```
<page>
  <place id=" ID ">
    <name>
      <text> 6 </text>
    </name>
    <graphics/>
    <type>
      <structure>
        <bool/>
      </structure>
    </type>
    <hlinitialMarking>
      <structure>
        <booleanconstant value="false"/>
      </structure>
    </hlinitialMarking>
    <toolspecific tool="Grafcet" version="IEC60848:2002">
      <OperatorDeclaration>
        <useroperator declaration=" continuousAction ">
          <variable refvariable=" Motor_1 ">
            <bool/>
          </variable>
        </useroperator>
      </OperatorDeclaration>
    </toolspecific>
  </place>
</page>
```

PLCopenXML (SFC)

```
<pou>
  <interface>
    <localVars retain="false">
      <variable name=" Motor_1 ">
        <type>
          <BOOL/>
        </type>
      </variable>
    </localVars>
  </interface>
  <body>
    <ST>
      <html>
        <div>
          STEP  S6 : Motor_1 ( N ); END_STEP
        </div>
      </html>
    </ST>
  </body>
</pou>
```

4.2. Tool support

None of the existing software tools which allow editing of a *Grafcet* offers an export function of the *Grafcet*. For this reason, the authors have spent efforts on the development of a GRAFCET-Editor which is suitable for automatically generating PLC code. The GRAFCET-Editor offers an interface to save the GRAFCET data in an XML file format that can further be processed.

When the control practitioner finishes working with the GRAFCET-Editor, he/she saves the *Grafcet*(*s*) as an XML file. The GRAFCET-Translator imports this XML file, analyses it and searches for GRAFCET relevant data, which is then stored in an internal object model. In subsequent steps, this object model is modified in accordance to the transformation concept presented in chapter 4. The export file generated by the GRAFCET-Translator is an XML file according to the PLCopenXML file format (Technical Paper, 2009), which is the standardized XML schema for IEC 61131-3. Additionally the GRAFCET-Translator provides the GRAFCET data as a PNML file.

Together, GRAFCET-Editor and GRAFCET-Translator form the tool framework for an automatic generation of IEC 61131-3 control code. The approach allows for an automatic generation of PLC code, which is compliant to the IEC 61131-3 standard's definitions, starting from a GRAFCET specification. Using the example of Table 3 the PNML file and the PLCopenXML file are shown in Table 4.

5. Application example

To demonstrate how GRAFCET is used within the systematic approach presented in the previous chapters an industrial size application example is examined in this chapter. The case study is a plant for quality control which conducts a stepwise functional testing of electromechanical parts as illustrated by the schematic diagram in Fig. 6. The plant consists of a conveyor belt and a rotary indexing table, each driven by one electro motor. The single test steps are carried out within the test stations 1, 2, 3, 5, 6 and 7 placed along the rotary indexing table. The motion sequence within the test stations is established by electro-pneumatic actuators, each having two limit position switches (front/ back stop position). Altogether, the plant possesses 61 digital input signals and 47 digital output signals which are controlled by a PLC.

The parts produced are transported to station 1 via the conveyor belt continuously. At station 1, as a first step, the parts are separated from each other and get placed onto the part fixation of the rotary indexing table across from station 1. The transfer is established by a picker arm. When the transfer is completed, the rotary indexing table moves in clockwise rotation to the next sector, e.g. until station 2 is reached. When rotation is completed test sequences within the test stations are processed. Consequently each part passes through the functional testing after seven process steps and gets labeled either as a regular or damaged part.

When designing the automation programs of this plant, several aspects of the plant and the technical process have to be considered in the design and specification of the control sequence, such as the modes of operation (automatic mode, manual mode) the operation phases (start-up, normal operation, shut-down) or safety issues. By using GRAFCET for the specification of the plant's control sequence, a systematic approach is now feasible. Therefore, in a bottom-up approach, the control sequence of each testing station is specified by one *partial Grafcet*, respectively. Then the plant's operation phases are considered within superior *partial Grafcets* which are logically interrelated to the testing stations' *partial Grafcets* by use of *enclosing steps* or *forcing orders*. Finally the plant's modes of operation are specified by

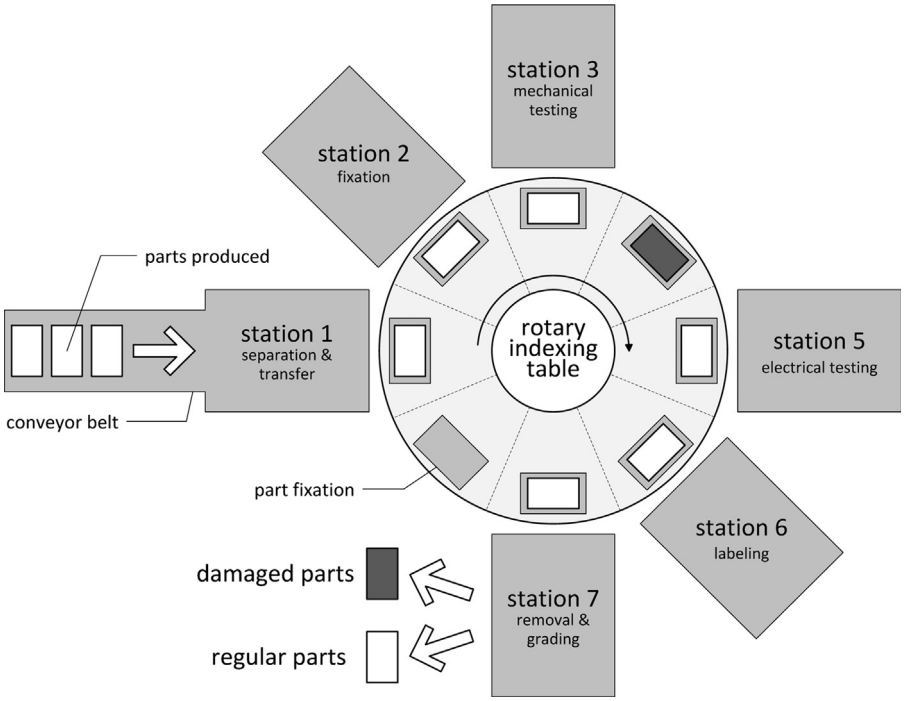


Fig. 6. Schematic diagram of the plant.

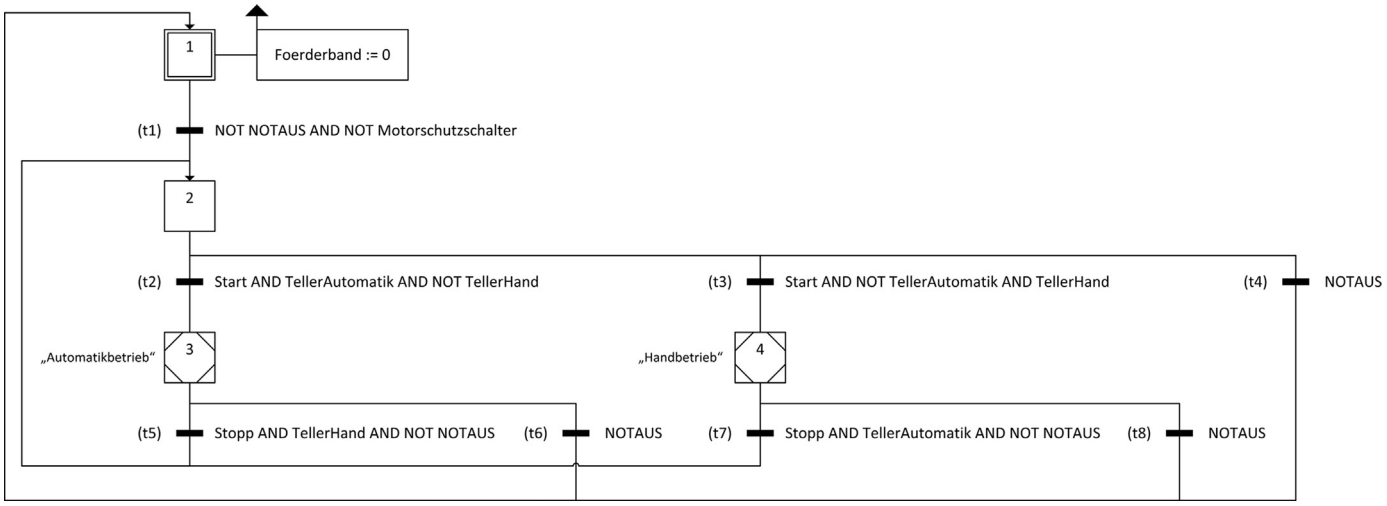


Fig. 7. Global Grafcet of the plant specifying the operation modes.

the *global Grafcet*. The logical interrelations between the *global Grafcet* and the *partial Grafcets* are again established by use of *enclosing steps* or *forcing orders*.

By reason of clarity the GRAFCET specification of the plant has been developed for the automatic mode and the normal operation phase only, neglecting visualization and safety aspects. The *Grafcet* consists of a three-stage hierarchical structure and is subdivided into one *global Grafcet* and seven *partial Grafcets*, altogether consisting of 60 steps, 62 transitions, 46 stored actions, 15 continuous actions and 8 *enclosing steps*. From this *Grafcet* the PLC code is generated by automatic transformation using the GRAFCET-Translator. The SFC structure of the PLC code consists of 61 steps, which is due to the constraint that in SFC only one initial step is allowed, and 109 transitions resulting from the normalization. Fig. 7 illustrates an excerpt of the plant's *global Grafcet* developed by use of the GRAFCET-Editor. The logical interrelations of the *Grafcet* were established by use of *enclosing steps*. On the example of the plant's *Grafcet*, the automatic generation of PLC code, which is compliant to IEC 61131-3 definitions, has been successfully performed. To allow readers to study the generated code in comparison to the GRAFCET specification in detail, the authors have placed specification and code at http://aut.hsu-hh.de/Grafcet-example.

Furthermore, the PLC code has been successfully imported to the engineering system CoDeSys (CoDeSys, 2013). CoDeSys is employed by many PLC manufacturers whose PLC hardware is thus a possible target platform for the automatically generated code.

6. Summary and outlook

Within this article the authors have presented a method for automatically generating PLC code from GRAFCET specifications

which is based on an exhaustive formal model of GRAFCET as Control Interpreted Petri Net. Especially the formalization of *enclosing steps*, *forcing orders* and *time constraints* allows for an unambiguous interpretation of the structure and the dynamic behavior of those significant elements which make GRAFCET such an expressive modeling language. Concerning this matter, the principle of normalization plays a key role. Since the IEC 61131-3 standard does not offer possibilities to consider hierarchy in the way IEC 60848 does in case of GRAFCET, normalization is a necessary step.

In future research work, the authors intend to enhance the current transformation method by retaining the hierarchical structure of the GRAFCET specification within the PLC code generated. Since structural information of a GRAFCET specification is also useful for the clarity of the resulting PLC code it is reasonable to develop a method which establishes an automatic transformation without normalization as a next step, however, this will require different means of representation in PLC code, beyond plain SFC. An additional analysis on how the presented approach can be applied to other PLC programming languages is considered worthwhile. In future work, the authors intend to develop a suitable transformation to Structured Text (ST) as an alternative to SFC. This offers higher computing speed and is comprehensible for informatics engineers. Furthermore, the authors' work will be focused on a detailed analysis of typical manual PLC code modifications and their appropriate redocumentation in GRAFCET, such as the modification of delay times within timed receptivities and the appropriate ascription in GRAFCET. The results of the corresponding investigations are to be the starting point towards a systematic bidirectional approach for the automatic generation of PLC code from a GRAFCET specification and the seamless redocumentation of code modifications.

Appendix A. Supporting information

Supplementary data associated with this article can be found in the online version at http://dx.doi.org/10.1016/j.conengprac.2014.09.008.

References

Alvarez, M. L., Burgos, A., Sarachaga, I., Estévez, E., Marcos, M.: GEMMA based approach for generating PLCopen Automation projects. In: *Proceedings of the IFAC conference on embedded systems, computational intelligence and telematics in control*, Wuerzburg, 03.-05.04.2012.

Arnold, G. V., Henriques, P. R.: A graphical interface based on GRAFCET for programming Industrial Robots off-line. In: *Proceedings of the international conference on informatics in control, automation and robotics*, Barcelona, 14.-17.09.2005.

Bauer, N., Huuck, R., Lukoschus, B., & Engell, S. (2004). A unifying semantics for sequential function charts. *Integration of Software Specification Techniques for Applications in Engineering, Lecture Notes in Computer Science*, *3147*, 400–418.

Carré-Ménétrier, V., & Zaytoon, J. (2002). Grafcet: behavioural issues and control synthesis. *European Journal of Control*, *8*(4), 375–401.

Charbonnier, F., Alla, H., David, R.: The supervised control of discrete event dynamic systems: a new approach, In: *Proceedings of the. IEEE conference on decision and control*, New Orleans 13.-15.12.1995.

Chériaux, F., Picci, L., Provost, J., Faure, J. M.: Conformance test of logic controllers of critical systems from industrial specifications. In: *Proceedings.of the European conference on safety and reliability*, Rhodos, 05.-09.09.2010.

CoDeSys - ⟨http://www.codesys.com/⟩ - last access: 06.12.2013.

David, R. (1995). Grafcet: a powerful tool for specification of logic controllers. *IEEE Transactions on Control Systems Technology*, *3*(3), 253–268.

David, R., & Alla, H. (2010). *Discrete, continuous and hybrid Petri nets*. Berlin: Springer-Verlag.

Deneux, H. (1983). *Contribution à l'étude des réseaux d' automates interconnectés* (Doctoral Thesis). Grenoble: Institut National Polytechnique (IPNG).

Fay, A., Biffl, S., Winkler, D., Drath, R., Barth, M.: A method to evaluate the openness of automation tools for increased interoperability. In: *Proceedings of the IECON 2013 – 39th annual conference of the IEEE Industrial electronics society*, Vienna,, 10.-13.11.2013.

Frensel, G., Bruijn, P. M.: From modeling control systems using Grafcet to analyzing systems using hybrid automata, In: *Proceedings of the American control conference*, Philadelphia, 21.-26.06.1998.

Gaertner, N., Thirion, B.: GRAFCET–an analysis pattern for event driven real-time systems, In: *Proceedings of the conference on pattern languages of programs*, Urbana, 15.-18.08.1999.

Guéguen, H., & Bouteille, N. (2001). Extensions of Grafcet to structure behavioral specifications. *Control Engineering Practice*, *9*(7), 743–756.

Hellgren, A., Fabian, M., & Lennartson, B. (2005). On the execution of sequential function charts. *Control Engineering Practice*, *13*(10), 1283–1293.

Hillah, L. M., Kindler, E., Kordon, F., Petrucci, L., & Trèves, N. (2009). A primer on the Petri Net markup language and ISO/IEC 15909-2. *Petri Net Newsletter*, *76*, 9–28.

IEC 60848, GRAFCET, specification language for sequential function charts, (3rd ed.), 2013.

IEC 61131-3, Programmable controllers—Part 3: Programming languages, (3rd ed.), 2013.

ISO/IEC 15909-2, Software and system engineering - High-level Petri nets - Part 2*:* *Transfer format*, 2011.

Johnson, T. L. (2007). Improving automation software dependability: a role for formal methods? *Control Engineering Practice*, *15*(11), 1403–1415.

L'Her, D., Le Parc, P., Marcé, L.: Modeling and proving Grafcets with transition systems. In: *Proceedings of the 2nd AMAST Workshop on real-time systems and project models and proofs*, Bordeaux, June 1995.

Le Parc, P., L'Her, D., Scharbarg, J.-L., & Marcé, L. (1999). Grafcet revisited with a synchronous data-flow language. *IEEE Transactions on Systems, Man and Cybernetics – Part A: Systems and Humans*, *29*(3), 284–293.

Lesage, J.-J., Roussel, J.-M., Thierry, C.: A theory of binary signal. In: *Proceedings of the IEEE multiconference on computational engineering in systems applications*, Lille, 09.-12.07.1996.

Lukman, T., Godena, G., Gray, J., Hericko, M., & Strmcnik, S. (2013). Model-driven engineering of process control software – beyond device-centric abstractions. *Control Engineering Practice*, *21*(8), 1078–1096.

Mallet, F., Gaffe, D., Boeri, F.: Concurrent control systems: from Grafcet to VHDL. In: *Proceedings of the 26th EUROMICRO conference*, Maastricht, 05.-07.09.2000.

Marcé, L., L'Her, D., Le Parc, P.: Modeling and verification of temporized Grafcet. In: *Proceedings of the IEEE multiconference on computational engineering in systems applications*, Lille, 09.-12.07.1996.

Martínez, M. A., & Martínez, J. L. (2005). Specification of operations for a manipulator on a mobile robot using grafcet. *Robotica*, *23*(6), 789–791.

Philippot, A., Tajer, A.: From GRAFCET to equivalent graph for synthesis control of discrete events systems. In: *Proceedings of the 18th Mediterranean conference on control & automation*, Marrakech, 23.-25.06.2010.

Provost, J., Roussel, J.-M., & Faure, J.-M. (2010). Translating Grafcet specifications into Mealy machines for conformance test purposes (doi 10.1016/j.conengprac.). *Control Engineering Practice*, *10*, 001.

Ramchandani, C. (1974). *Analysis of asynchronous concurrent systems by yimed Petri nets* (PhD Thesis). Cambridge: Massachusetts Institute of Technology.

Roussel, J.-M., Lesage, J.-J.: Validation and verification of GRAFCETs using finite state machine. In: *Proceedings of the IEEE multiconference on computational engineering in systems applications*, Lille, 09.-12.07.1996.

Schumacher, F., Fay, A.: Requirements and obstacles for the transformation of GRAFCET graphs into IEC 61131-3 PLC programs. In: *Proceedings of the IEEE emerging technologies & factory automation*, Toulouse, 2011.

Schumacher, F., Fay, A.: Transforming time constraints of a GRAFCET graph into a suitable Petri net formalism. In: *Proceedings of the IEEE international conference on industrial technology*, Cape Town, 25.-27.02.2013.

Schumacher, F., Schröck, S., Fay, A.: Transforming hierarchical concepts of GRAFCET into a suitable Petri net formalism. In: *Proceedings of the IFAC conference on manufacturing modeling, management and control*, Saint Petersburg, 19.-21.06.2013.

Technical Paper, PLCopen Technical Committee 6 (TC6): XML Formats for IEC 61131-3, Version 2.01 - Official Release, 2009.

Thramboulidis, K., & Frey, G. (2011). Towards a model-driven IEC 61131-based development process in industrial automation. *Journal of Software Engineering and Applications*, *4*(4), 217–226.

Witsch, D., Vogel-Heuser, B.: Close integration between UML and IEC 61131-3: New possibilities through object-oriented extension. In: *Proceedings of the IEEE Emerging Technologies & Factory Automation*, Mallorca, 22.-26.09.2009.

World Wide Web Consortium (W3C), Extensible Markup Language (XML) 1.0 (Fifth Edition), retrieved 5/2012, URL: ⟨http://www.w3.org/TR/xml/⟩.